

Autonomous Salvage ROVs in the Hadal Research Base “Abyssus”

FREE CHOICE ASSIGNMENT FOR THE COURSE *AUTOMATED PLANNING*, UNIVERSITY OF TRENTO, A.Y. 25/26

Stefano Racca,
stefano.racca@unitn.it

Giorgia Lorengo
giorgia.lorenco@unitn.it

1 Introduction

This document aims to describe the main challenges encountered during the crafting of the final solution for the **Free Choice Scenario** of the course *Automated Planning: Theory and Practice* at the University of Trento. Specifically, we will describe how we transformed the assignment into code, how we coped with domain-specific requirements of the solvers, and how the resolution of each task differ from previous ones.

All the code and results obtained are available at the **GitHub repo** <https://github.com/Stefanoo01/Automated-Planning.git>.

2 Scenario Description

2.1 Motivation

The proposed scenario is set in *Abyssus*, a fictional deep-sea research facility located in the Mariana Trench. The scenario was designed to model a realistic robotic planning problem while introducing heterogeneous agents, constrained navigation, and specialized sample-handling procedures.

The choice of this environment allows classical, multi-agent, hierarchical, temporal, and execution-oriented planning techniques to be explored within a single coherent domain.

2.2 Environment

The Abyssus facility is divided into several interconnected sectors dedicated to underwater research and sample preservation. Autonomous Remotely Operated Vehicles (ROVs) are responsible for recovering scientific samples scattered throughout the station and transporting them to the Bio-Vault, where they are permanently stored.

The environment contains both standard and pressure-sensitive samples, which requires slightly different handling procedures.

Figure 1 illustrates the main sectors of the facility

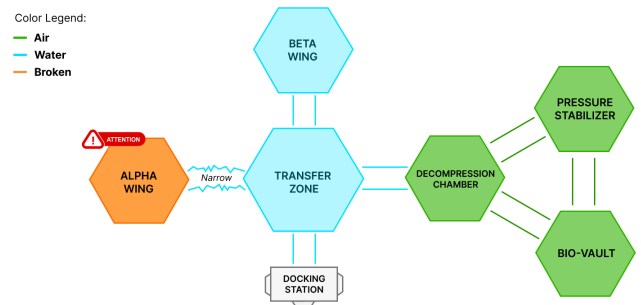


Figure 1: Schematic representation of the Abyssus environment.

and the connections between them. Part of the base operates underwater, including the transfer zone and Wings Alpha and Beta, whereas the operations and storage areas are located in a dry section of the facility. These two sections are connected through a decompression chamber. The ROVs can navigate between locations by traversing the corridors that connect the different sectors.

A recent catastrophic event has compromised one sector of the Abyssus Base, Wing Alpha. The main corridor connecting this sector to the rest of the facility has become hazardous and difficult to traverse. Consequently, Wing Alpha can only be accessed through narrow passages that are inaccessible to larger robots. This constraint introduces heterogeneous navigation capabilities and motivates the need for coordination among multiple agents in the later tasks.

Moreover, the facility contains a docking station where robots can recharge their batteries whenever necessary.

2.3 Goal State

The objective of the mission is to safely recover all scientific samples and store them in the Bio-Vault. Standard samples only need to be collected and delivered, whereas pressure-sensitive samples must first be enclosed in protective capsules and subsequently stabilized using a dedicated pressure stabilizer before

final storage.

Depending on the specific task, the generated plan may additionally need to minimize action costs, respect temporal constraints, exploit multiple agents, or be executable through PlanSys2.

2.4 Initial State

The planning problem starts with one or more ROVs positioned at the facility docking station. Scientific samples are initially distributed between the Wing Alpha and Wing Beta sectors, while empty protective capsules are made available for handling pressure-sensitive samples.

Initially, all ROVs have a fully charged battery and no sample has been collected, stabilized, or stored in the Bio-Vault. Depending on the task, robots may differ in carrying capacity and operational capabilities.

3 Domain Design

The domain was designed incrementally, starting from a classical single-agent planning model and progressively extending it with multi-agent coordination, hierarchical decomposition, temporal constraints, and execution in PlanSys2. The same core scenario was preserved across all tasks, while the representation was adapted when required by solver-specific limitations.

3.1 Types

The main entities of the domain are **locations**, **ROVs**, **samples**, and **capsules**:

- **Locations** represent the different sectors of the Abyssus facility, including the Docking Station, Wing Alpha, Wing Beta, the Pressure Stabilizer, and the Bio-Vault.
- **ROVs** are the autonomous agents responsible for navigating the facility and transporting objects.
- **Samples** are divided into regular **samples** and **pressure-sensitive** samples. **Pressure-sensitive** samples are modelled as a subtype of **sample** because they share the same final objective, but require additional handling before storage.
- **Capsules** are locatable objects used to safely transport **pressure-sensitive** samples.

3.2 Predicates

The domain predicates can be grouped into four main categories:

1. **Spatial predicates** - such as **at**, **connected**, and **narrow-connected** - describe the topology of the facility and the position of robots, samples, and capsules.
2. **Robot-state predicates** - such as **carrying** - describe the state of a ROV, mainly whether it is managing and/or transporting an object.
3. **Sample-state predicates** - such as **regular-sample**, **stabilized**, and **stored** - distinguish sample categories and record their progress toward the mission goal.
4. **Capsule-related predicates** - such as **empty-capsule**, **sample-in-capsule**, and **capsule-sealed** - model the additional safety procedure required for pressure-sensitive samples.

3.3 Actions

The action model is organized around a small set of operational capabilities. Navigation is handled through standard movements (implemented via the **move** action) between connected locations, with an additional **move-through-narrow** action introduced in the multi-agent version. Sample handling is modeled through the **pick-up**, **drop**, and **store** actions for regular samples, as well as dedicated variants for the handling of pressure-sensitive samples. Moreover, for the management of the latter, there are also additional actions such as **take-empty-capsule**, **encapsulate-sample**, and **stabilize-capsule**.

Resource management is represented through battery levels, action costs, and robot capacity. In the classical and temporal domains, battery and cost are represented using numeric fluents. In the HTN domain, however, capacity is represented discretely through predecessor relations, which are implemented using types, in order to remain compatible with PANDA.

3.4 Assumptions

The original specification intentionally left several implementation details unspecified. The following assumptions were adopted throughout all tasks:

- Pressure-sensitive samples must be stabilized before final storage.
- Narrow passages can only be traversed by robots explicitly marked as small.
- Protective capsules can be reused after a sample has been delivered.

These assumptions remained unchanged throughout all tasks, ensuring a consistent domain specifica-

tion while allowing different planning paradigms and solvers to be compared under the same operational setting.

3.5 Design Choices

One of the main modelling decisions was to represent pressure-sensitive samples as a subtype of sample rather than only as a predicate. This allowed the domain to express common behaviour for all samples while restricting specific actions, such as encapsulation and stabilization, to sensitive samples only.

Capsules were introduced to make the handling of pressure-sensitive samples more realistic and to create additional dependencies between actions.

Another relevant design choice concerns agent heterogeneity. From Task 2 onward, ROVs may differ in both carrying capacity and navigation capabilities. In particular, narrow passages prevent larger robots from traversing certain areas, encouraging cooperative plans. This allows, in the HTN model, to explicitly represent these constraints and enable the implementation of handover strategies, allowing robots to cooperate and reducing the overall plan duration.

Finally, some representations were adapted to solver requirements: `:numeric-fluents` was used in ENHSP, OPTIC, and TFD to model battery and costs, while `:fluents` was used in POPF since it does not support numeric-fluents. Instead the HTN model, as previously stated, used a discrete representation of capacity because of PANDA compatibility constraints.

4 Evolution of the model

4.1 Task 1: Classical Planning

4.1.1 Implementation Details

The first task models the Abyssus scenario as a classical single-agent planning problem. The objective was to validate the core domain by introducing all the fundamental interactions required by the mission while keeping the planning model as simple as possible.

To increase the realism of the planning problem, even if not explicitly required by the assignment, a numeric battery model was introduced to represent the limited autonomy of the vehicle. Each movement or manipulation action decreases the battery level, while the `recharge` action restores it at the Docking Station. This forces the planner to periodically schedule recharge operations.

Action costs were also associated with every operator and optimized via the `total-cost` metric. In particular, movement and unnecessary intermediate manipulations were assigned relatively high costs in order to discourage inefficient plans without explicitly forbidding them.

4.1.2 Experimental Results

Since the `problem.pddl` file is solver-dependent, instead of testing different solvers (and therefore developing multiple problem files), we evaluated the classical planning model using **ENHSP** with different heuristic search strategies. The choice to use this solver was mainly supported by the fact that it was the only one - among the ones tested - that consistently generated plans for the more complex problem instances during the early phases of development.

The goal for this section was to identify the most efficient configuration in terms of planning time required by the solver to generate a valid plan, as well as the quality of the resulting plan measured by the number of actions required. In particular, the ENHSP solver was used and **tested with both satisfiability and optimality heuristics**. The satisfiability setting is expected to produce a plan more quickly, although it may be suboptimal. In contrast, the optimality setting aims to generate the optimal plan, but at the cost of increased computation time. In particular, the heuristics considered are ¹:

- SAT-HADD - Greedy Best First Search with numeric hadd.
- SAT-AIBR - AStar plus AIBR heuristic.
- OPT-HMAX - AStar with hmax numeric heuristic.
- OPT-BLIND - baseline blind heuristic that gives 1 to states where the goal is not satisfied and 0 to states where the goal is satisfied.

To evaluate the different strategies, three different problem files have been generated, with increased complexity:

- EASY: includes a ROV, a regular sample, a pressure-sensitive sample, and one capsule.
- MEDIUM: extends the EASY problem by adding one regular and one pressure-sensitive object, while leaving the number of capsule unchanged.
- HARD: extends the MEDIUM problem by adding one regular and one pressure-sensitive object, while leaving the number of capsule unchanged.

Notice that the MEDIUM and the HARD problem both have a single capsule to exploit the ability of the solver to reuse the same capsule for multiple

¹Source: <https://sites.google.com/view/enhsp/home/how-to-use-it>

Table 1: Results with different ENHSP heuristics.

Problem	Heuristic	# Actions	$\tilde{t}_P$ (s)	σ_{tP} (s)
EASY	opt-blind	23	1.41	0.13
	opt-hmax	23	2.07	0.12
	sat-aihr	23	8.08	0.68
	sat-hadd	23	1.13	0.13
MEDIUM	opt-blind	47	7.73	0.74
	opt-hmax	47	34.95	2.50
	sat-aihr	N/A	N/A	N/A
	sat-hadd	73	1.86	0.04
HARD	opt-blind	N/A	N/A	N/A
	opt-hmax	N/A	N/A	N/A
	sat-aihr	N/A	N/A	N/A
	sat-hadd	107	2.15	0.23

samples.

Table 1 reports the average planning time $\tilde{t}_P$ - computed over five different executions - required by each ENHSP heuristic for the different problem instances. σ_{tP} represents the variability of the planning time measurements and is computed as the standard deviation over the five executions.

As expected, increasing the complexity of the problem significantly affects the performance of the heuristics. Several search strategies are unable to produce a solution within a reasonable amount of time as the problem size grows². Nevertheless, whenever a solution is returned, the generated plan always satisfies the mission objectives and respects all domain constraints. In particular, pressure-sensitive samples are encapsulated, stabilized, and stored in the correct order, whereas regular samples are directly delivered to the Bio-Vault.

In general, the planner prioritizes regular samples before pressure-sensitive ones, as they can satisfy part of the goal with fewer actions. It also correctly manages numeric resources by scheduling recharge actions before the battery becomes depleted, and by recovering and reusing capsules for subsequent pressure-sensitive samples.

For the EASY problem, both satisfiability and optimality oriented heuristics compute an optimal solution within a short amount of time. Among them, **hadd** achieves the lowest planning time and therefore represents the most effective configuration. Notably, it is also the only heuristic capable of solving the HARD problem within the imposed time limit.

As the problem complexity increases, the performance of satisfiability heuristics rapidly deteriorates.

²The evaluation script interrupts the solver after five minutes if no solution has been found. This limit was introduced for evaluation purposes and because some heuristics were observed to exhaust the available memory while searching for a plan.

For example, **aihr** is unable to solve the MEDIUM problem within the allotted five-minute limit, while optimality heuristics maintains relatively low planning times while producing significantly shorter plans (47 versus 73 actions). Inspection of the plans generated by satisfiability heuristics reveals the source of their inefficiency: several pressure-sensitive samples are repeatedly dropped and picked up at intermediate locations before their final delivery. Although the resulting plans are valid, these unnecessary manipulations increase both the number of actions and the amount of robot movement without contributing directly to the mission objectives.

Among the optimality heuristic, even if the number of actions required by the plan is the same, **opt-hmax** uses approximately five times the time needed to compute the solution, marking **opt-blind** the best candidate for up to medium problems.

4.2 Task 2: Multi-Agent Coordination

4.2.1 Implementation Details

Task 2 extends the classical planning model by introducing multiple cooperating ROVs: small ROVs have a carrying capacity of one object, but they are the only agents allowed to traverse narrow passages; larger ROVs, instead, cannot access restricted areas but can transport multiple objects simultaneously thanks to their higher carrying capacity. As a result, the planner must coordinate the different agents to efficiently complete the mission.

Supporting multiple carrying capacities also required a modification of the object manipulation model. In the single-agent domain, the **handempty** predicate was sufficient to ensure that the robot could carry at most one object at a time. Since larger ROVs are now allowed to transport multiple objects simultaneously, this predicate was removed and replaced by a numeric **capacity** fluent, which is increased whenever an object is released and decreased whenever an object is picked up.

To support the new coordination mechanisms, the domain additionally introduces the **:disjunctive-preconditions** and **:conditional-effects** PDDL requirements. The remaining handling procedures are preserved from Task 1, as well as battery consumption and action costs, that are maintained using **:numeric-fluents**.

4.2.2 Experimental Results

As in Task 1, this second part of the assignment has been evaluated on **ENHSP** using different heuristic search strategies.

Table 2: Results with different ENHSP heuristics.

Problem	Heuristic	# Actions	$\tilde{t}_P$ (s)	σ_{tP} (s)
EASY	opt-blind	18	1.61	0.16
	opt-hmax	18	1.72	0.09
	sat-aibr	20	3.78	0.21
	sat-hadd	21	1.73	0.10
MEDIUM	opt-blind	32	43.40	1.48
	opt-hmax	32	126.11	6.18
	sat-aibr	N/A	N/A	N/A
	sat-hadd	23	282.84	5.11
HARD	opt-blind	32	9.48	0.67
	opt-hmax	32	19.66	1.20
	sat-aibr	34	121.01	5.26
	sat-hadd	38	102.41	3.78

To evaluate each strategy, three different problem files have been generated, with increased complexity³:

- EASY: includes a 1-capacity ROV, a 3-capacity ROV, one standard sample, one pressure-sensitive sample, and one capsule.
- MEDIUM: extends the EASY problem by adding one additional pressure-sensitive object and one additional capsule.
- HARD: extends the MEDIUM problem by removing one capsule.

Table 2 summarizes the average planning time and plan quality obtained with each ENHSP heuristic.

Inspection of the generated *.plan* files reveals several interesting behaviors. The plan produced for the EASY problem demonstrates effective cooperation between the two ROVs by exploiting the smaller robot’s ability to traverse the narrow passage, while the larger robot is used to store the collected samples. However, likely due to the limited number of samples, the planner does not fully exploit the larger robot’s storage capacity. Instead, it makes the robot performs multiple trips to the storage area rather than waiting until more samples have been transferred so they can be transported together.

In the MEDIUM problem, which introduces an additional pressure-sensitive object, a clear divergence emerges between plans generated using optimality-based and satisficing heuristics. The optimal planners continue to rely primarily on *rov1*, whereas the satisficing planners employ both ROVs. However, in the latter case, *rov2* repeatedly traverses the envi-

ronment without transporting or storing any objects, resulting in unnecessary movements that do not contribute to accomplish the task. A similar behavior is also observed in the HARD problem.

One possible explanation is the optimization metric, namely *total-cost*. Although the move action has the highest cost, the generated plans appear to prioritize satisfying intermediate subgoals over fully exploiting the larger ROV’s carrying capacity.

4.3 Task 3: Hierarchical Planning

4.3.1 Implementation Details

Task 3 reformulates the Abyssus domain as a Hierarchical Task Network (HTN) planning problem. The initial task network consists of the abstract task *complete-mission*, which is recursively decomposed into the processing of all regular and pressure-sensitive samples. Two recursive tasks, *process-all-sensitive-samples* and *process-all-regular-samples*, select one pending sample, process it, mark it as completed, and continue until no samples remain.

Navigation is also represented hierarchically through the abstract task *navigate-to*. Depending on the connectivity, it is decomposed either into a direct movement or into a recursive sequence through intermediate locations. A *visited* predicate prevents navigation loops, while narrow passages are handled separately through *move-through-narrow*.

To exploit heterogeneous robot capabilities, the domain includes handover methods. For samples located behind narrow passages, a small ROV first transports the sample or capsule to the transfer zone, where another ROV can continue the mission. Since PANDA and Lilotane do not support numeric fluents, battery management was removed and carrying capacity was discretized using *capacity-number* objects. Auxiliary actions such as *mark-*-done*, *visit*, and *unvisit* were introduced to manage recursion and task progression.

4.3.2 Experimental Results

The HTN model was evaluated using both PANDA and Lilotane. Without handovers, PANDA solved the domain in 0.082 seconds, producing a plan with 170 primitive actions. Introducing explicit handover methods forces cooperation between different ROVs and results in more realistic executions, although at the cost of increased search complexity. In this version, PANDA required 9.337 seconds and 181 actions, while Lilotane solved the same instance in 12.281 seconds using 217 actions.

³We intended to evaluate the planners on more complex problem instances involving a larger number of samples in order to investigate whether cooperation between the two ROVs would become more effective. However, the resulting planning problems proved to be too computationally demanding, and none of the considered heuristic was able to solve them within a reasonable amount of time.

Table 3: HTN scalability results averaged over five seeds.

Problem	Planner	# Actions	$\tilde{t}_P$ (s)	σ_{tP} (s)
P1	PANDA	45.8	1.88	0.03
	Lilotane	54.0	0.30	0.01
P2	PANDA	72.2	2.58	0.51
	Lilotane	84.8	0.27	0.02
P3	PANDA	85.2	2.31	0.54
	Lilotane	102.4	1.07	0.11
P4	PANDA	114.4	12.34	19.98
	Lilotane	139.2	0.65	0.07
P5	PANDA	127.8	2.36	0.12
	Lilotane	162.4	3.53	0.29
P6	PANDA	144.6	2.62	1.08
	Lilotane	186.0	2.01	0.29
P7	PANDA	171.6	42.20	86.67
	Lilotane	215.2	15.91	5.74

To evaluate the scalability of the HTN model, the handover domain was tested on seven increasingly complex problems by progressively increasing the number of regular and pressure-sensitive samples. Each instance was executed using five different random seeds, and the reported values correspond to the average results.

Both planners successfully solved every benchmark. PANDA consistently generated shorter plans than Lilotane, requiring between 45.8 and 171.6 primitive actions on average, whereas Lilotane required between 54.0 and 215.2 actions for the same instances.

Table 3 reports the complete numerical results, while Figure 2 illustrates how the average number of primitive actions evolves as the problem size increases. Both planners exhibit an approximately linear growth in plan length as additional samples are introduced, confirming the expected increase in mission complexity. However, the gap between the two curves remains fairly constant across all instances, showing that PANDA consistently finds more compact decompositions than Lilotane. The small variations around the general trend are mainly caused by different task decompositions and robot assignments selected under different random seeds.

Planning time showed a much less regular behaviour and did not increase monotonically with the number of samples. This suggests that HTN planning complexity depends not only on the problem size, but also on the decomposition choices and search strategy adopted by each planner.

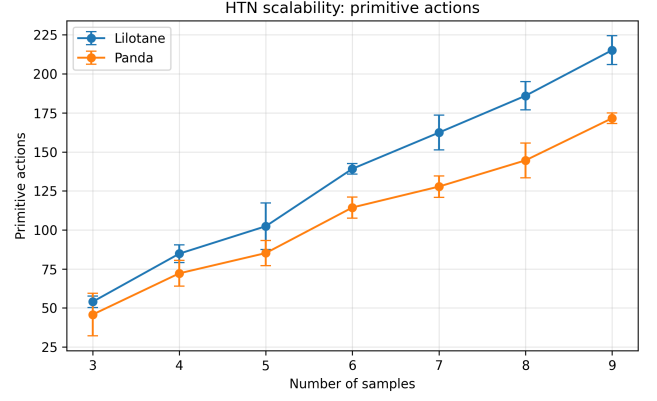


Figure 2: Average number of primitive actions over five seeds.

4.4 Task 4: Temporal Planning

4.4.1 Implementation Details

Task 4 extends the multi-agent domain introduced in Task 2 by reformulating it as a temporal planning problem. Instead of considering actions as instantaneous events, the temporal model associates a duration with every operator, allowing multiple robots to execute compatible activities concurrently. This enables the planner to exploit parallelism whenever possible, potentially reducing the overall execution time while preserving all the constraints already introduced in the previous tasks.

The temporal domain was **evaluated using the OPTIC and the TFD planners**. Although both support durative actions, they differ in the subset of PDDL they accept. Consequently, two slightly different versions of the domain (and, consequently, of the corresponding problem) were developed while preserving the same operational semantics.

The first modification consisted in replacing instantaneous actions with durative ones by introducing the `:durative-actions` requirement and assigning an execution time to every operator. Temporal conditions were then distributed among the `at start`, `over all`, and `at end` sections according to the intended behaviour. In particular, actions involving the manipulation of objects, such as picking up samples or encapsulating pressure-sensitive ones, require both the robot and the manipulated object to remain at the same location for the entire action duration. These invariants prevent unrealistic executions in which a robot could begin manipulating an object and move away before the action has been completed.

Several additional modifications were required to ensure compatibility with both planners. Since neither OPTIC nor TFD fully supports `:negative-preconditions`,

:disjunctive-preconditions,
:quantified-preconditions, or
:conditional-effects in the considered configuration, these constructs were removed from the temporal model. In particular, the original :negative-preconditions used to prevent repeated stabilization of pressure-sensitive samples was replaced by the explicit predicate `unstabilized`. Although this slightly increases the number of predicates in the domain, it preserves the intended semantics while remaining compatible with both planners.

The representation of specific facility locations was also revised: instead of modelling the docking station and the pressure stabilizer as dedicated object types, they are identified through the predicates `is-docking-station` and `is-pressure-stabilizer`. This formulation proved to be more robust across the considered temporal planners, avoiding grounding and type-matching issues while maintaining the same expressive power.

Finally, minor syntactic adaptations were required because of parser-specific requirements. OPTIC expects the :numeric-fluents requirement instead of the more general :fluents declaration accepted by TFD, and it also requires the declaration of functions to appear before the type definitions.

4.4.2 Experimental Results

The solvers tested for Task 4 are OPTIC and TFD which, as previously stated, require a slightly different PDDL code to generate a plan.

Although both solvers achieve the same goals, the resulting plans - reported in Table 4 - differ significantly in terms of quality, parallelism, and resource utilization. The main differences are summarized below.

- **Execution of actions.** OPTIC schedules independent actions concurrently whenever possible. For example, while `rov1` moves toward Wing Alpha, `rov2` simultaneously retrieves an empty capsule and heads toward Wing Beta. Later, both rovers return to the storage area in parallel, significantly reducing the overall makespan.
- **Division of work.** In the OPTIC plan, to each rover is assigned a well-defined role: `rov1` is responsible for collecting and storing the regular sample from Wing Alpha, whereas `rov2` collects both the regular and the sensitive samples from Wing Beta, stabilizes the capsule, and stores both samples. This allocation minimizes unnecessary travel and exploits the capabilities of each rover effectively.

Table 4: Results comparing TFD and OPTIC solvers.

Problem	# Actions	Plan Duration (s)	$\tilde{t}_P$ (s)	σ_{tP} (s)
OPTIC	20	40.00	0.69	0.08
TFD	29	75.00	50.53	1.86

In contrast, TFD processes the sensitive sample first, while the collection of the regular samples is postponed until much later. As a result, the specialization of the two rovers is lost. Furthermore, `rov1` performs several unnecessary trips, while `rov2` - which has sufficient capacity to retrieve both samples in a single trip - actually takes and store one sample at a time. These additional round trips account for most of the extra actions in the plan.

Overall, the TFD plan is considerably less efficient, producing a more sequential and movement-intensive solution, with a plan that contains significantly more actions (29 versus 20) and has a much longer makespan (75 seconds compared with 40 seconds). Instead, OPTIC generates plans with substantially better temporal efficiency for this domain.

4.5 Task 5: PlanSys2 and ROS2

4.5.1 Implementation Details

A more realistic execution framework was developed in Task 5 using the PlanSys2 framework within the ROS2 ecosystem. Each action defined in Task 4 was implemented as a simulated durative action through dedicated C++ modules, providing a realistic approximation of the execution of the generated plans.

The planner adopted in this task is POPF, the default planner integrated into PlanSys2. Similar to OPTIC and TFD, POPF does not support the :negative-preconditions requirement. Consequently, the PDDL domain and problem definitions developed for Task 4 could be reused with only minor modifications, such as replacing the ‘-’ character with ‘_’ since not supported by the planner.

Furthermore, unlike the planners used previously, PlanSys2 does not directly accept a PDDL problem file as input. Instead, it requires a plain-text specification containing the list of objects to instantiate and the goal to be achieved (which, only later, is internally converted into a PDDL problem before invoking the planner). To automate this conversion, the Python script `pddl_parser.py` was developed to parse a PDDL problem file and generate the corresponding input format required by PlanSys2.

The entire project was built using CMake through the provided `CMakeLists.txt` configuration and in-

tegrated with the ROS2 build system.

4.5.2 Execution and Reproducibility

The code for Task5 is the only one that has a dedicated Docker file, which can be found in `task5/Dockerfile` and can be launched via the bash script `task5/start_docker.sh`. To manage the execution, you will need two open terminals, but all the instructions to run the simulation can be found in `task5/readme.md`, as well as the output logs obtained.